

Experiment No. 1

Aim

To understand the use of **NumPy** and **Matplotlib** for creating a simple chart and setting the properties of a plot.

Theory

NumPy is a Python library used for numerical operations and arrays. **Matplotlib** is used for graphical representation and data visualization. Using Matplotlib, we can set different plot properties such as title, axis labels, markers, grid, and legend.

Program

```
import numpy as np
import matplotlib.pyplot as plt

# Creating data using NumPy
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# Creating a simple chart
plt.plot(x, y, marker='o', label='y = 2x')

# Setting plot properties
plt.title("Simple Chart using Matplotlib")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid(True)
plt.legend()

# Save the chart
plt.savefig("experiment1_output.png")

print("Graph saved successfully!")
```

Output

A simple line chart is displayed with **title**, **X-axis** and **Y-axis** labels, **markers**, **grid**, and **legend**.

Result

Thus, a simple chart was successfully created using **NumPy and Matplotlib**, and different plot properties were applied successfully. 

Experiment No. 2

Aim

To understand the use of **Matplotlib** for working with **multiple figures and axes**, and adding **text, grid, and legend** to plots.

Theory

Matplotlib allows us to create multiple plots using **figures and axes**. We can customize plots by adding text, grid lines, labels, titles, and legends.

Program

```
import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]
y1 = [2, 4, 6, 8, 10]
y2 = [1, 4, 9, 16, 25]

# Create multiple axes
fig, ax = plt.subplots(1, 2, figsize=(10, 4))

# First plot
ax[0].plot(x, y1, marker='o', label='y = 2x')
ax[0].set_title("First Plot")
ax[0].set_xlabel("X Values")
ax[0].set_ylabel("Y Values")
ax[0].grid(True)
ax[0].legend()

# Second plot
ax[1].plot(x, y2, marker='s', label='y = x²')
ax[1].set_title("Second Plot")
ax[1].set_xlabel("X Values")
ax[1].set_ylabel("Y Values")
ax[1].grid(True)
ax[1].legend()
```

```
# Adding text
ax[1].text(2, 20, "Sample Text")

plt.tight_layout()

# Save output
plt.savefig("experiment2_output.png")

print("Graph saved successfully!")
```

Output

The program creates **two plots** with:

- Multiple axes
- Text
- Grid
- Legend
- Titles and axis labels

The output is saved as **experiment2_output.png**.

Result

Thus, multiple plots and axes were successfully created using **Matplotlib**, and text, grid, and legend were added successfully. 

Experiment No. 3

Aim

To understand the use of **Matplotlib** for creating different types of charts such as **Line Chart**, **Histogram**, **Bar Chart**, and **Pie Chart**.

Theory

Matplotlib is a Python library used for data visualization. Different functions are used to create different charts:

- `plot()` – Line Chart
- `hist()` – Histogram
- `bar()` – Bar Chart
- `pie()` – Pie Chart

Program

```
import matplotlib.pyplot as plt

# Line Chart
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.figure()
plt.plot(x, y, marker='o')
plt.title("Line Chart")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid(True)
plt.savefig("line_chart.png")

# Histogram
data = [10, 20, 20, 30, 30, 30, 40, 40, 50]

plt.figure()
plt.hist(data, bins=5)
plt.title("Histogram")
plt.xlabel("Values")
```

```
plt.ylabel("Frequency")
plt.savefig("histogram.png")

# Bar Chart
subjects = ["C", "Python", "Java", "ML"]
marks = [80, 85, 75, 90]

plt.figure()
plt.bar(subjects, marks)
plt.title("Bar Chart")
plt.xlabel("Subjects")
plt.ylabel("Marks")
plt.savefig("bar_chart.png")

# Pie Chart
plt.figure()
plt.pie(marks, labels=subjects, autopct="%1.1f%%")
plt.title("Pie Chart")
plt.savefig("pie_chart.png")

print("All charts saved successfully!")
```

Output

The program creates and saves:

- line_chart.png
- histogram.png
- bar_chart.png
- pie_chart.png

Result

Thus, **Line Chart, Histogram, Bar Chart, and Pie Chart** were successfully created using **Matplotlib**. 

Experiment No. 4

Aim

To implement **Linear Regression** in Python for **house price prediction** using the Boston Housing dataset.

Theory

Linear Regression is a supervised machine learning algorithm used to predict continuous numerical values. In this experiment, the model uses housing-related features to predict the target variable **MEDV**, which represents the median house value. ([GitHub](#))

Program

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = pd.read_csv("Boston.csv")

# Input features and target
X = data.drop("medv", axis=1)
y = data["medv"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Evaluate the model
```

```
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

print("\nActual Prices:")
print(list(y_test[:5]))

print("\nPredicted Prices:")
print(list(y_pred[:5]))
```

Output

The program displays:

- **Mean Squared Error (MSE)**
- **R² Score**
- Actual house prices
- Predicted house prices

Result

Thus, the **Linear Regression model** was successfully implemented for **house price prediction** using the Boston Housing dataset. 

Experiment No. 5

Aim

To implement the **K-Nearest Neighbor (KNN) classifier** in Python for **diabetes classification** using the Pima Indians Diabetes Dataset.

Theory

K-Nearest Neighbor (KNN) is a supervised machine learning algorithm used for classification. It predicts the class of a data point based on the majority class of its **K nearest neighbors**.

In this experiment, the model predicts:

- 0 → No Diabetes
- 1 → Diabetes

Requirements

Make sure your `experiment5` folder contains:

```
experiment5/  
├── 5.py  
└── diabetes.csv
```

Install required libraries inside your virtual environment:

```
pip install pandas matplotlib scikit-learn
```

Program

```
import pandas as pd  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.neighbors import KNeighborsClassifier  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import accuracy_score, confusion_matrix,  
ConfusionMatrixDisplay  
  
# Load dataset  
data = pd.read_csv("diabetes.csv")  
  
# Separate input features and target  
X = data.drop("Outcome", axis=1)
```

```
y = data["Outcome"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("KNN Accuracy:", round(accuracy * 100, 2), "%")

# Create Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Visualize Confusion Matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Diabetes", "Diabetes"]
)
```

```
)

disp.plot()

plt.title("KNN Diabetes Classification")
plt.savefig("knn_confusion_matrix.png")

print("\nVisualization saved as knn_confusion_matrix.png")
```

Output

The program displays:

- **KNN Accuracy**
- **Confusion Matrix**

It also creates a visual output file:

knn_confusion_matrix.png

You can open this image to see the **visual classification results**. 

Result

Thus, the **K-Nearest Neighbor (KNN) classifier** was successfully implemented for **diabetes classification**, and the model performance was visualized using a **Confusion Matrix**. 

Experiment No. 6

Aim

To build a **Naive Bayes model** in Python for classifying SMS messages as **Spam or Ham**.

The SMS Spam Collection dataset contains messages labeled as ham (legitimate) or spam; the relevant columns are typically v1 for the label and v2 for the message text. ([Kaggle](#))

Theory

Naive Bayes is a supervised machine learning algorithm based on probability. It is commonly used for **text classification**, including spam detection.

- **Ham** → Normal/legitimate message
- **Spam** → Unwanted message

Dataset

Download the dataset from:

[SMS Spam Collection Dataset – Kaggle](#)

After downloading, place the file as:

spam.csv

inside your experiment6 folder.

The downloaded CSV commonly contains extra unnamed columns, so the program below selects only v1 and v2. ([Vonage Developer](#))

Program

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Load dataset (TSV format)
data = pd.read_csv(
    "spam.csv",
    sep="\t",
    names=["label", "message"])
```

```

)

# Convert labels into numerical values
data["label"] = data["label"].map({"ham": 0, "spam": 1})

# Input and target
X = data["message"]
y = data["label"]

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Convert text into numerical features
vectorizer = CountVectorizer()

X_train_vector = vectorizer.fit_transform(X_train)
X_test_vector = vectorizer.transform(X_test)

# Create Naive Bayes model
model = MultinomialNB()

# Train model
model.fit(X_train_vector, y_train)

# Prediction
y_pred = model.predict(X_test_vector)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Naive Bayes Accuracy:", round(accuracy * 100, 2), "%")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

```

```
print("\nConfusion Matrix:")
print(cm)

# Visualize Confusion Matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Ham", "Spam"]
)

disp.plot()

plt.title("Naive Bayes Spam Classification")
plt.savefig("naive_bayes_confusion_matrix.png")

print("\nVisualization saved as naive_bayes_confusion_matrix.png")
```

Requirements

Install the required libraries inside your `.venv-1`:

```
pip install pandas matplotlib scikit-learn
```

Output

The program displays:

- **Naive Bayes Accuracy**
- **Confusion Matrix**

It also creates:

naive_bayes_confusion_matrix.png



This image will visually show how accurately the model classified **Ham and Spam** messages.

Result

Thus, the **Naive Bayes model was successfully implemented for SMS Spam Classification**, and the results were visualized using a **Confusion Matrix**. 

Experiment No. 7

Aim

To implement a **Decision Tree classifier** in Python for classifying wine into **three different types**.

Theory

A **Decision Tree** is a supervised machine learning algorithm used for classification. It divides data into branches based on different features and predicts the class of new data.

In this experiment, the Wine dataset is classified into:

- Wine Type 0
- Wine Type 1
- Wine Type 2

Program

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Load the Wine dataset
data = pd.read_csv(
    "9408623-b237fa5848349a14a14e5d4107dc7897c21951f5/wine.csv"
)

# Display dataset information
print("Dataset Shape:", data.shape)
print("\nColumns:")
print(data.columns)

# Input features and target
X = data.iloc[:, 1:]
y = data.iloc[:, 0]
```

```
# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Decision Tree model
model = DecisionTreeClassifier(random_state=42)

# Train the model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nDecision Tree Accuracy:", round(accuracy * 100, 2), "%")

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Visualize Decision Tree
plt.figure(figsize=(16, 8))

plot_tree(
    model,
    feature_names=X.columns.astype(str),
    filled=True
)

plt.title("Decision Tree for Wine Classification")
plt.tight_layout()
```

```
plt.savefig("decision_tree_wine.png")

print("\nDecision Tree saved as decision_tree_wine.png")

# Visualize Confusion Matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.title("Wine Classification Confusion Matrix")
plt.savefig("wine_confusion_matrix.png")

print("Confusion Matrix saved as wine_confusion_matrix.png")
```

Output

The program displays:

- **Decision Tree Accuracy**
- **Confusion Matrix**

It also creates two visual output files:

decision_tree_wine.png

wine_confusion_matrix.png

Result

Thus, the **Decision Tree classifier** was successfully implemented to classify wine into **three different types**, and the results were visualized using a **Decision Tree and Confusion Matrix**. 

Experiment No. 8

Aim

To implement a **Support Vector Machine (SVM)** classifier in Python for **diabetes classification** using the **Pima Indians Diabetes Dataset**.

Theory

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification. It finds the best decision boundary (hyperplane) that separates different classes.

In this experiment:

- 0 → No Diabetes
- 1 → Diabetes

We will use the same `diabetes.csv` dataset from **Experiment 5**.

Program

Copy `diabetes.csv` from Experiment 5 into the `experiment8` folder.

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Load dataset
data = pd.read_csv("diabetes.csv")

# Input features and target
X = data.drop("Outcome", axis=1)
y = data["Outcome"]

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Scale the data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create SVM model
model = SVC(kernel="linear")

# Train the model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("SVM Accuracy:", round(accuracy * 100, 2), "%")

# Create Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Visualize Confusion Matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Diabetes", "Diabetes"]
)

disp.plot()

plt.title("SVM Diabetes Classification")
plt.savefig("svm_confusion_matrix.png")
```

```
print("\nVisualization saved as svm_confusion_matrix.png")
```

Terminal Commands

Create the Experiment 8 folder:

```
cd "/home/nayan153/Desktop/ML experiment"  
mkdir -p experiment8  
cd experiment8
```

Copy the dataset from Experiment 5:

```
cp "../experiment5/diabetes.csv" .
```

Check files:

```
ls
```

You should see:

```
8.py  
diabetes.csv
```

Run the program:

```
python 8.py
```

Output

The program displays:

- **SVM Accuracy**
- **Confusion Matrix**

It also creates:

```
svm_confusion_matrix.png
```

Result

Thus, the **Support Vector Machine (SVM)** classifier was successfully implemented for **diabetes classification**, and its performance was visualized using a **Confusion Matrix**. 

Experiment No. 9

Aim

To demonstrate the application of an **Artificial Neural Network (ANN)** using Python for a classification problem.

Theory

An **Artificial Neural Network (ANN)** is inspired by the structure of the human brain. It consists of interconnected neurons arranged in layers:

- **Input Layer** – Receives input data
- **Hidden Layer** – Processes and learns patterns
- **Output Layer** – Produces the final prediction

For this experiment, we will use the **Iris dataset** to classify flowers into three different classes.

Program

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

# Load Iris dataset
data = load_iris()

# Input features and target
X = data.data
y = data.target

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Scale the data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create Artificial Neural Network model
model = MLPClassifier(
    hidden_layer_sizes=(10, ),
    max_iter=1000,
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("ANN Accuracy:", round(accuracy * 100, 2), "%")

# Create Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Visualize Confusion Matrix
disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=data.target_names
)
```

```
disp.plot()

plt.title("ANN Iris Classification")

# Save visualization
plt.savefig("ann_confusion_matrix.png")

print("\nVisualization saved as ann_confusion_matrix.png")
```

Terminal Commands

Create the Experiment 9 folder:

```
cd "/home/nayan153/Desktop/ML experiment"
```

```
mkdir -p experiment9
```

```
cd experiment9
```

Create your Python file:

```
nano 9.py
```

Paste the code, save it, then run:

```
python 9.py
```

Output

The program displays:

- **ANN Accuracy**
- **Confusion Matrix**

It also creates:

ann_confusion_matrix.png



You can open this image to see the ANN classification performance visually.

Result

Thus, the application of an **Artificial Neural Network (ANN)** was successfully demonstrated using Python for the **classification of Iris flowers into three different classes**. 